SIMATS ENGINEERING

Assessment Tool 3 — Dry Run Test

Course: Artificial Intelligence (CSA17) | CO2: Dry Run Testing (BL3)

Question 1: Dry Run of A* Search Algorithm (10 Marks)

Graph edges and weights: A-B=2, A-C=4, B-C=3, B-D=7, B-E=2, C-E=3, D-E=2, E-G=2.

Heuristic values: h(A)=7, h(B)=6, h(C)=4, h(D)=3, h(E)=2, h(G)=0.

Goal: find the path from A to G using f(n) = g(n) + h(n).

Dry Run Trace

Iter.	Current Node	Open List	Closed List	g(n)	h(n)	f(n)
1	A	{ B, C }	{ }	0	7	7
2	B	{ C, D, E }	{ A }	2	6	8
3	E	{ C, D(updated), G }	{ A, B }	4	2	6
4	G (Goal)	{ C, D }	{ A, B, E }	6	0	6

Notes on the trace:

- Iteration 1: A is expanded → B (g=2,h=6,f=8) and C (g=4,h=4,f=8) are added to Open.
- Iteration 2: B has the lower g among the tied f=8 nodes, so it is expanded next → D (g=9,f=12) and E (g=4,f=6) are added; C's g via B (5) is worse than existing (4), so it is not updated.
- Iteration 3: E has the lowest f=6, so it is expanded → G (g=6,f=6) is added; D is updated from g=9 to g=6 (better path via E); C via E (g=7) is worse than existing (4), not updated.
- Iteration 4: G has the lowest f=6 and is the goal, so the search terminates.

Final Answers

1. Optimal Path: A → B → E → G

2. Total Path Cost: 2 + 2 + 2 = 6

Question 2: Dry Run of Minimax with Alpha-Beta Pruning (10 Marks)

Tree: MAX root has two MIN children. Left MIN's leaves = 3, 5, 6. Right MIN's leaves = 9, 1, 2. Evaluated strictly left to right.

Dry Run Trace

Step	Node Visited	Alpha (α)	Beta (β)	Value / Action
1	MAX (root) — enter	-∞	+∞	Begin, go to MIN(L)
2	MIN(L) leaf 3	-∞	3	β = min(∞,3) = 3
3	MIN(L) leaf 5	-∞	3	β = min(3,5) = 3 (no change)
4	MIN(L) leaf 6	-∞	3	β = min(3,6) = 3 (no change)
5	MIN(L) returns	-∞	3	value = 3
6	MAX (root) update	3	+∞	α = max(-∞,3) = 3; go to MIN(R)
7	MIN(R) leaf 9	3	9	β = min(∞,9) = 9
8	MIN(R) leaf 1	3	1	β = min(9,1) = 1; α(3) ≥ β(1) → PRUNE
9	MIN(R) leaf 2	—	—	PRUNED — never evaluated
10	MIN(R) returns	3	1	value = 1
11	MAX (root) update	3	+∞	α = max(3,1) = 3
12	MAX (root) returns	3	+∞	Final value = 3

Explanation:

- Left MIN node: all three leaves (3, 5, 6) must be evaluated since α stays at -∞ throughout, so no pruning is possible there; the node returns the minimum value, 3.
- Back at MAX root, α is updated to 3 (the best value found so far for MAX).
- Right MIN node inherits α = 3. After leaf 9 (β=9) and leaf 1 (β=1), the condition α ≥ β becomes 3 ≥ 1, which is TRUE — so the third leaf (value 2) is pruned and never evaluated.
- Right MIN node returns 1. MAX root compares 3 and 1, and keeps 3 as the final value.

Final Answers

1. Best Move for MAX: Choose the LEFT subtree (MIN-left), which yields value 3.

2. Final Minimax Value: 3

3. Pruned Nodes: The third leaf of the right MIN node (value = 2) is pruned.

Appendix A: Python Code — A* Search (User-Defined / Dynamic Input)

The program below is not hard-coded to this graph. It lets the user enter any number of edges, any node names, any weights, and any heuristic values at runtime, then performs the same iteration-by-iteration A* dry run shown above (verified to reproduce the exact trace: A → B → E → G, cost 6).

```
"""
A* Search Algorithm - Dynamic (User-Defined) Input Version
------------------------------------------------------------
Accepts ANY graph (nodes, edges, weights) and heuristic values
entered by the user at runtime, then performs a full dry run,
printing Open List, Closed List, g(n), h(n), f(n) at every step.
"""

import heapq


def get_graph_input():
    graph = {}
    n = int(input("Enter number of edges: "))
    print("Enter each edge as:  NodeA NodeB Weight   (e.g.  A B 2)")
    for _ in range(n):
        u, v, w = input("Edge: ").split()
        w = float(w)
        graph.setdefault(u, {})[v] = w
        graph.setdefault(v, {})[u] = w
    return graph


def get_heuristics(nodes):
    h = {}
    print("\nEnter heuristic value h(n) for every node:")
    for node in nodes:
        h[node] = float(input(f"  h({node}) = "))
    return h


def a_star(graph, heuristics, start, goal):
    open_list = [(heuristics[start], 0, start, [start])]
    closed = set()
    g_score = {start: 0}
    iteration = 1

    while open_list:
        f, g, current, path = heapq.heappop(open_list)

        print(f"\n--- Iteration {iteration} ---")
        print(f"Current Node : {current}")
        print(f"g(n) = {g}   h(n) = {heuristics[current]}   f(n) = {f}")
        print(f"Open List    : {[item[2] for item in open_list]}")
        print(f"Closed List  : {sorted(closed)}")
        iteration += 1

        if current == goal:
            print(f"\nGoal '{goal}' reached!")
            print(f"Optimal Path : {' -> '.join(path)}")
            print(f"Total Cost   : {g}")
            return path, g

        closed.add(current)
        for neighbor, weight in graph.get(current, {}).items():
            if neighbor in closed:
                continue
```

```
            tentative_g = g + weight
            if neighbor not in g_score or tentative_g < g_score[neighbor]:
                g_score[neighbor] = tentative_g
                f_score = tentative_g + heuristics[neighbor]
                heapq.heappush(open_list, (f_score, tentative_g, neighbor, path + [neighbor]))

    print("No path found.")
    return None, None


if __name__ == "__main__":
    print("=== A* Search (User-Defined Graph) ===")
    graph = get_graph_input()
    nodes = list(graph.keys())
    heuristics = get_heuristics(nodes)
    start = input("\nEnter start node: ").strip()
    goal = input("Enter goal node: ").strip()
    a_star(graph, heuristics, start, goal)
```

Appendix B: Python Code — Minimax with Alpha-Beta Pruning (User-Defined / Dynamic Input)

The program below accepts any game tree (any depth or branching factor) as a nested list entered by the user, then performs the same alpha-beta dry run shown above, printing every alpha/beta update and reporting pruned nodes (verified to reproduce the exact trace: final value 3, pruned node = right MIN's third leaf).

```
"""
Minimax with Alpha-Beta Pruning - Dynamic (User-Defined) Input Version
----------------------------------------------------------------------
Accepts ANY game tree entered by the user (as a nested list, so any
branching factor / depth is supported), then performs a full dry run,
printing alpha, beta, selected values and pruned branches at every node.
Root is always assumed to be a MAX node, levels alternate MIN/MAX.
"""

import math
import ast


def build_tree_input():
    print("Enter the game tree as a nested Python list.")
    print("Example (matches the assignment):  [[3, 5, 6], [9, 1, 2]]")
    print("(Each inner list = children of a node; numbers = leaf values.)")
    tree_str = input("Tree: ")
    return ast.literal_eval(tree_str)


def minimax_ab(node, depth, alpha, beta, maximizing, path="root"):
    indent = "  " * depth

    # Leaf node
    if isinstance(node, (int, float)):
        print(f"{indent}Leaf {path}: value = {node}")
        return node, []

    pruned = []

    if maximizing:
        value = -math.inf
        print(f"{indent}MAX node [{path}]  (alpha={alpha}, beta={beta})")
        for i, child in enumerate(node):
            if alpha >= beta:
                pruned.append(f"{path}.{i}")
                print(f"{indent}  -> Child {path}.{i} PRUNED (alpha {alpha} >= beta {beta})")
                continue
            child_val, child_pruned = minimax_ab(child, depth + 1, alpha, beta, False, f"{path}.{i}")
            pruned.extend(child_pruned)
            value = max(value, child_val)
            alpha = max(alpha, value)
            print(f"{indent}  Updated at {path}: alpha={alpha}, beta={beta}, best so far={value}")
        print(f"{indent}MAX node [{path}] returns {value}")
        return value, pruned

    else:
        value = math.inf
        print(f"{indent}MIN node [{path}]  (alpha={alpha}, beta={beta})")
        for i, child in enumerate(node):
            if alpha >= beta:
                pruned.append(f"{path}.{i}")
                print(f"{indent}  -> Child {path}.{i} PRUNED (alpha {alpha} >= beta {beta})")
                continue
            child_val, child_pruned = minimax_ab(child, depth + 1, alpha, beta, True, f"{path}.{i}")
```

```
            pruned.extend(child_pruned)
            value = min(value, child_val)
            beta = min(beta, value)
            print(f"{indent}  Updated at {path}: alpha={alpha}, beta={beta}, best so far={value}")
        print(f"{indent}MIN node [{path}] returns {value}")
        return value, pruned


if __name__ == "__main__":
    print("=== Minimax with Alpha-Beta Pruning (User-Defined Tree) ===")
    tree = build_tree_input()
    result, pruned_nodes = minimax_ab(tree, 0, -math.inf, math.inf, True)
    print(f"\nFinal Minimax Value : {result}")
    print(f"Pruned Nodes         : {pruned_nodes if pruned_nodes else 'None'}")
```